Media Downloader

Building, installing and using the app, its engine and the browser extension.
Revised 2026-08-16.

What this is

A private desktop downloader for YouTube, Reddit, Twitter/X, HDRezka and roughly 1800 other sites. Nothing is uploaded, no account is needed, and every setting stays on your machine.

It is three pieces that ship together:

Piece	Technology	What it does
Shell	Electron	The window and interface
Engine	Python	yt-dlp, tagging, file sorting
Extension	Chrome MV3	Reads HDRezka pages in your browser

The shell talks to the engine over a private pipe, and the engine talks to the extension over a token-protected port on 127.0.0.1. Only the interface is web technology; the downloading is all Python.

1. What you need first

Requirement	Version	Why
Python	3.10 or newer	Runs the download engine
Node.js and npm	18 or newer	Runs and packages the shell
ffmpeg	any recent	Merges video with audio, converts audio
Google Chrome or Edge	any recent	Only needed for HDRezka

Check what you already have:

```
python --version
node --version
ffmpeg -version
```

Note. If `python` opens the Microsoft Store, install Python from python.org instead and tick **Add python.exe to PATH**. The Store build is sandboxed and PyInstaller cannot always read from it.

2. Installing

Engine dependencies

```
cd "D:\GitHub Repos\MediaDownloaderProject"
python -m pip install -r requirements.txt
```

Shell dependencies

```
cd desktop
npm install
```

Note. npm 11 and newer block install scripts, so Electron's runtime download is skipped and starting it fails with *Electron failed to install correctly*. Fix it with `npm approve-scripts electron` followed by a reinstall. If that still fails, download `electron-v<version>-win32-x64.zip` from the Electron releases page, extract it into `node_modules/electron/dist/`, and put the text `electron.exe` into `node_modules/electron/path.txt`.

3. Running from source

This is the normal way to use it during development:

```
cd desktop
npm start
```

The window opens and the engine starts automatically as a child process. You never launch the engine yourself. Add `-- --dev` to open developer tools.

4. Building executables

There are two builds. The engine is frozen with PyInstaller, then the shell is packaged with electron-builder and the engine is bundled inside it.

Step one: freeze the engine

```
cd "D:\GitHub Repos\MediaDownloaderProject"
python -m PyInstaller Engine.spec --noconfirm ^
--distpath dist-engine --workpath build-engine
```

Produces `dist-engine\mediadl-engine.exe`. It is a console-less program that speaks JSON over its input and output, so double-clicking it does nothing visible. That is expected.

Step two: package the shell

```
cd desktop
npm run dist
```

Produces an installer and a portable build in `dist-desktop\`. electron-builder copies `dist-engine` into the app's resources, and the shell prefers that frozen engine over your Python install when it finds it.

Optional: the icon

```
python tools\make_icon.py
```

Regenerates `mediadl\resources\app.ico` from the logo at seven sizes. Only needed if you change the mark.

Build reference

Command	Output
`python -m PyInstaller Engine.spec`	dist-engine\mediadl-engine.exe
`npm run dist`	dist-desktop\ installer and portable
`python tools\make_icon.py`	mediadl\resources\app.ico
`python tools\make_manual.py`	docs\Media-Downloader-Manual.pdf

Note. A running copy locks its own executable, so a rebuild fails with a permission error that looks unrelated. Close the app first. The PowerShell build script does this for you.

5. Installing the browser extension

Only needed for HDRezka. The site refuses requests that do not come from a real browser, so the page is read inside yours; the extension is the bridge between that page and the app.

1. Open `chrome://extensions` and turn on **Developer mode** (top right).
2. Click **Load unpacked** and choose the `extension` folder in the project.
3. Start the app, then go to **Download** → **HDRezka**. It shows an address and a pairing token.
4. Click **Copy token**, then open the extension's **Options**, paste it, and press **Save and test**.

5. You should see *Connected*. The app's HDRezka tab turns green too.

Note. The token is what stops any web page you visit from talking to the app. It is generated once and kept in your settings folder. The extension only ever contacts 127.0.0.1, never the internet.

6. Using it

Downloading from YouTube and most sites

1. Go to the **Download** tab and pick **Auto detect** or **YouTube**.
2. Paste one or more links, one per line. Playlists and channels expand into separate queue items on their own.
3. Choose the mode, quality and container.
4. Set the destination folder, then press **Add to queue**.

The view switches to **Queue**, where each item shows progress, speed and remaining time, with pause, resume, retry and reveal-in-folder controls.

The window

The app draws its own titlebar, so the window controls sit at the right of the top bar rather than in a separate strip. Drag the empty area to move the window, and double-click it to maximise. **F11** toggles fullscreen and **Esc** leaves it; the middle button changes to an inward-arrow icon while fullscreen, and pressing it returns to a normal window.

Downloading from HDRezka

1. Open the film or series page in your browser and leave the tab open.
2. In the app, go to **Download** → **HDRezka** and press **Read open HDRezka tab**.
3. Pick the translation (dub) and quality. Both come from the page itself.
4. Tick what you want in the season tree. Each season expands to its episodes, and the season checkbox selects the whole season.
5. Press **Download**. Progress appears in the **Queue** tab.

Selection shortcuts in the tree:

- **Select all** ticks every episode of every season.
- **Clear** unticks everything.
- The range box accepts `1-10` or `3,5,7`, and applies to whichever seasons are expanded, or all of them if none are.
- A part-selected season shows a dash rather than a tick, with a `3 / 21` count beside it.

Note. HDRezka files are fetched by the browser, so they land in your Downloads folder first and the app moves them to the real destination when each one finishes. Chrome will not let an extension write anywhere else. Nothing is left behind; the staging folder is a waypoint.

Size checks and batching

Before anything starts, one episode is measured and multiplied by how many you selected. The figure is approximate but close, because episodes of a series are similar sizes.

That total is compared against free space on the drive the browser downloads to. If a whole season would not fit, the work is split into batches automatically: a batch downloads, those files are moved to the destination, and only then does the next batch start. The staging drive therefore never holds more than one batch at a time. The Queue heading shows which batch is running and how many of its files have been filed.

What the queue is telling you

A browser download passes through several stages, and each one is named in the queue so a slow step is never mistaken for a broken one:

Stage	Meaning
Downloading in browser	Chrome is fetching the file. Progress is real.
Waiting for the browser to release the file	The transfer finished but Chrome still holds the file. The app is checking it has stopped growing and matches the expected size.
Moving to destination	Copying from the Downloads folder to your chosen folder. Progress is real. A large file across drives takes a while; this is normal.
Writing tags	Adding title, show, season and episode metadata.
Completed	The file is in place and verified.

Note. A file is only ever deleted from the staging folder after the copy has been verified byte for byte. If a move fails the original is left untouched, so nothing is lost. A half-written file is never left where a finished one should be.

Where files end up

Series are filed by show and season:

```
D:\Movies\House M.D\
House M.D.\
Season 06\
House M.D. 6x20 LostFilm.mp4
House M.D. 6x21 LostFilm.mp4
```

The name pattern is `show season x episode dub`. The dub is dropped when unknown, and the show prefers the page's original title when it has one. Change the pattern in **Settings**; a broken pattern falls back to a safe default rather than failing.

Settings worth knowing

Setting	What it does
Simultaneous downloads	How many run at once. Three is a sensible default.
Season folders	Turn off to keep every episode in one flat folder.
Expand playlists	Off means a playlist becomes one item, not many.
Use cookies from	For age-restricted or members-only videos.

Download in chunks	Leave on. Prevents long transfers dying at HTTP 403.
Strict container matching	Leave off. A known cause of HTTP 403 on YouTube.

7. When something goes wrong

The **Logs** tab shows what the app and yt-dlp actually did, filterable by level and text. The same content is written to a rotating file:

```
%APPDATA%\MediaDownloader\mediadl.log
```

Common problems

Symptom	Cause and fix
Requested format is not available on a video that plays fine	yt-dlp is out of date. Run `python -m pip install --upgrade yt-dlp` and rebuild the engine.
Every download fails right after enabling cookies	Chrome locks its cookie database while running. Close Chrome, or set **Use cookies from** back to None. The app warns and carries on without them.
HTTP 403 partway through a large download	The media URL expired. It retries automatically. Keep **Download in chunks** on and **Strict container matching** off.
Could not establish connection from the extension	The content script is not in that tab. Reload the extension at chrome://extensions, then reload the HDRezka tab once.
This does not look like a HDRezka title page	You are on a listing or search page rather than a title, or the site changed its markup.
HDRezka returns an anti-bot page	Open the title in your browser and let its check pass, then use the extension. The app cannot get past it on its own, by design.
Nothing downloads and the engine dot is red	The engine did not start. Check Python is on PATH and the requirements are installed.
A download sits on *Moving to destination* for a long time	It is copying between drives, which is a real copy rather than a rename. The percentage moves; leave it. Nothing is deleted until the copy is verified.
Size mismatch in the log and the file was not filed	The browser transfer ended early. The partial file is deliberately left in the staging folder rather than filed as if it were complete. Download that episode again.

Where things live

What	Where
Settings	`%APPDATA%\MediaDownloader\settings.json`
Log file	`%APPDATA%\MediaDownloader\mediadl.log`
Pairing token	`%APPDATA%\MediaDownloader\bridge-token.txt`
History	`%APPDATA%\MediaDownloader\history.json`

Browser staging	`Downloads\MediaDownloader\`

8. Keeping it working

Sites change their players constantly, so yt-dlp goes stale faster than anything else here. When downloads that used to work start failing, update it first:

```
python -m pip install --upgrade yt-dlp
```

Then rebuild the engine so the packaged app picks it up. The version in use is shown in the app's **Settings** panel under About.

Download only what you have the right to download.